

1. Deploy your your previous assignment flask frontend and express backend in kubernetes cluster locally with minikube.

Submission guidelines: share github repo link and screenshots of your running deployments and commands.

Answer

To deploy your Flask frontend and Express backend applications locally using Minikube, follow this step-by-step implementation guide. Since we are running this locally, we will use Minikube's built-in Docker daemon to build the images directly inside the cluster, avoiding the need to push them to Docker Hub.

1. Prerequisites & Project Structure

Ensure your project repository is organized clearly. Here is a standard structural layout for your GitHub repository:

my-k8s-app/

├── frontend/

| ├── Dockerfile

| └── [Flask App Files]

├── backend/

| ├── Dockerfile

| └── [Express App Files]

└── k8s/

└── frontend-deployment.yaml

└── backend-deployment.yaml

2. Kubernetes Manifests (k8s/)

Backend Deployment & Service (backend-deployment.yaml)

This configuration manages your Express backend. It exposes the app internally within the cluster using a ClusterIP service.

apiVersion: apps/v1

kind: Deployment

metadata:

name: express-backend

labels:

app: backend

spec:

replicas: 1

selector:

matchLabels:

app: backend

template:

metadata:

labels:

app: backend

spec:

containers:

- name: backend

image: express-backend:local

imagePullPolicy: Never # Forces Minikube to use the locally built image

ports:

- containerPort: 3000

apiVersion: v1

kind: Service

metadata:

name: backend-service

spec:

selector:

app: backend

ports:

- protocol: TCP

port: 3000

targetPort: 3000

type: ClusterIP

Frontend Deployment & Service (**frontend-deployment.yaml**)

This configuration manages the Flask frontend. It exposes the application outside the cluster using a **NodePort** service

apiVersion: apps/v1

kind: Deployment

metadata:

name: flask-frontend

labels:

app: frontend

spec:

replicas: 1

selector:

matchLabels:

app: frontend

template:

metadata:

labels:

app: frontend

spec:

containers:

- name: frontend

image: flask-frontend:local

imagePullPolicy: Never

ports:

- containerPort: 5000

env:

- name: BACKEND_URL

value: "http://backend-service:3000" # Inter-pod communication via CoreDNS

apiVersion: v1

kind: Service

metadata:

name: frontend-service

spec:

selector:

app: frontend

ports:

- protocol: TCP

port: 5000

targetPort: 5000

nodePort: 30080 # Accessible at http://<minikube-ip>:30080

type: NodePort

3. Step-by-Step Deployment Commands

Execute the following commands in your terminal to spin up the cluster and deploy the microservices:

Step 1: Start Minikube

```
minikube start
```

Step 2: Point your terminal to Minikube's Docker Daemon

Important: This step tells your local Docker CLI to build images directly inside Minikube's internal Docker registry. Run this in the terminal window where you plan to build your images.

```
eval $(minikube docker-env)
```

Step 3: Build the Docker Images

Navigate to your project directories and build the images locally:

```
# Build Express Backend
```

```
docker build -t express-backend:local ./backend
```

```
# Build Flask Frontend
```

```
docker build -t flask-frontend:local ./frontend
```

Step 4: Apply Kubernetes Manifests

Apply the configuration files to your cluster:

```
kubectl apply -f k8s/backend-deployment.yaml
```

Step 5: Verify the Deployments

Check if your pods, deployments, and services are up and running successfully:

```
kubectl get all
```

Step 6: Access the Frontend Application

To open your web application in your local browser, ask Minikube to fetch the URL for the NodePort service:

```
minikube service frontend-service --url
```

4. Submission Checklist

To finalize your assignment submission, compile your deliverables matching these guidelines:

- **GitHub Repository Link:** Provide the URL to your public or accessible project repository containing your application code, Dockerfiles, and the `k8s/` configuration manifests.
- **Required Screenshots:**
 1. **Terminal Output:** A screenshot showing the result of running `kubectl get all` or `kubectl get pods`, proving that both the Flask and Express pods are in the `Running` state.

2. **Running Application:** A screenshot of your browser showing the frontend user interface loading successfully via the Minikube-provided local URL.